LINEAR REGRESSION - 2

Linear algo

Agenda ⇒ → Recap → scikit Learn code

(1) Model Interpretability ⇒

(2) feature importance

(3) GD Gradient Descent → LR

Train & Test ⇒ 3 (4 parts

⇒ Train, VALIDATING, Testing

model

model → 4.56

Retune hyperparams

$$\hat{y} = 4.67 w_1 + 6.57$$

Loss → MAE, MSE

Model Interpretability

Model eqⁿ

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + w_0 \times 1$$

w_0 = Skip

$\Rightarrow$ age of car

$$\hat{y} = \cdots\cdots + (-10000)\cdot \text{age} + (-10)\,\text{odometer} + (200)\,\text{engine} + \cdots$$

wt. for age; wt. for odometer; wt.

wt. $\Rightarrow$ -ve value?

Case I) w_j = -ve

$x_i \uparrow \Rightarrow$ price will $\downarrow$

INVERSE Relation

odometer $\uparrow \Rightarrow$ Resale price $\downarrow$

2026

2021

age

5 years

Reduced

age ↑ 1 year ⇒ ^ 10 k units

(+)

(b) weights / coeff. ⇒ these have meaning

sign ⇒ ↑ or ↓ feature value

what happens ?

case II) coeff. / w_j = +ve

car1 → 1100 CC

engine

car2 → 1200 cc → ↑ price

w_j ⇒

↑ engine 1 unit ⇒ Price ↑ 200 units

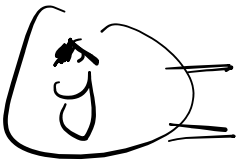
-ve ⇒ ↓

+ve ⇒ ↑

-ve ⇒ ↓ +ve ⇒ ↑



Case III) coeff. of $w_j = 0$

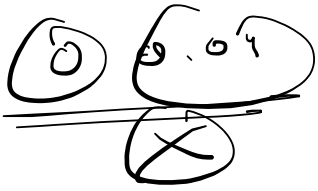
$wt. = 0$ ⇒ 

no impact ✗

prod. ⇒ 

(*) $wt = 0$ keep?

Feature selection Techniq



+ve → Direct prop

-ve → inversely prop

(+) INTERPRETABILITY ⇒

(+) INTERPRETABILITY ⇒

* FEATURE importance ⇒

feature → weights

(+) Compare wts of features

→ magnitude of wt. (w_j) ↑ ⇒ Higher importance of (x_j)

(+)

$\hat{y} = (0.8)x_2 + \cdots + (0.45)x_5$

x_2 ⇒ more impact

1 unit ↑ in x_2 ⇒ 0.8 units ↑ or ↓ $\hat{y}$

$\hat{y} = (-1.9)x_2 + \cdots + (-4.4)x_5$

$\hat{y} = (-1.9)x_2 + \cdots + (-4.1)x_5$

$x_5 \rightarrow$ High impact

$x_5 \uparrow$ 1 unit $\Rightarrow$ price $\downarrow$ 4.4 units

$\hat{y} = (2.3)x_2 + \cdots + (-5.1)x_5$

$\Rightarrow x_5 \rightarrow$ 1 unit $\Rightarrow$ price -5.1

interpret & understand importance

$\Rightarrow \hat{y} = \cdots + \text{age} + \cdots$

years

(-10000)

$(1 - 15)$

small

Similar scale

$(-10) \rightarrow$ odometer $+$ so on

50k, 100k ---

model training

Similar scale

X

small

model training

idea for scaling

$w \times x \Rightarrow$

Similar scale

* Price = w_1 AREA + w_2 BedRooms

AREA ↓ 500-5000

BedRooms 1-5

X

model learned → Trained

Price = 200(AREA) + 50000(BedRooms)

$w \times x$

5 lk > 200

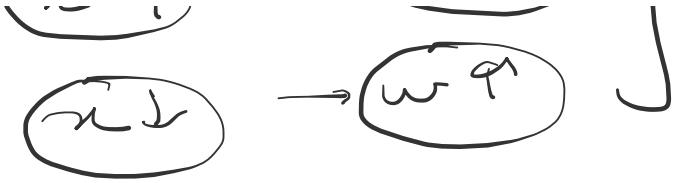
X

$w \times x$

$x \uparrow \rightarrow w \downarrow$

$\rightarrow w \uparrow$

models



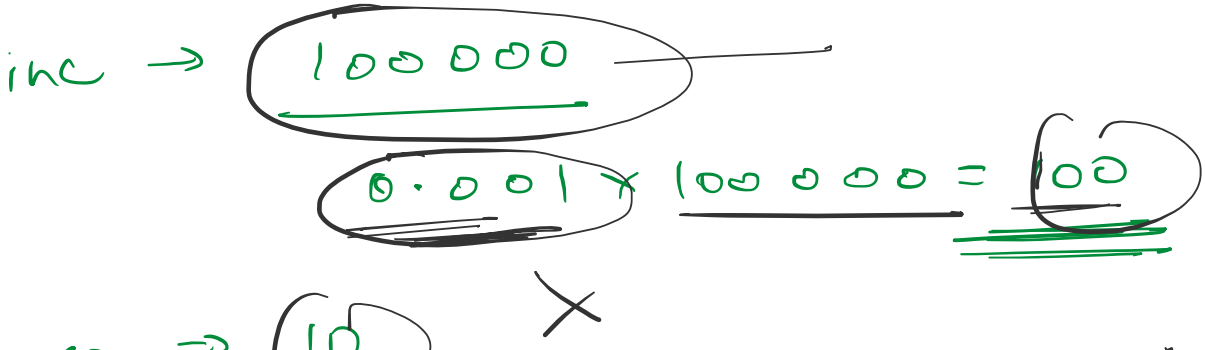
Features →

inc — 0 – 1 CR

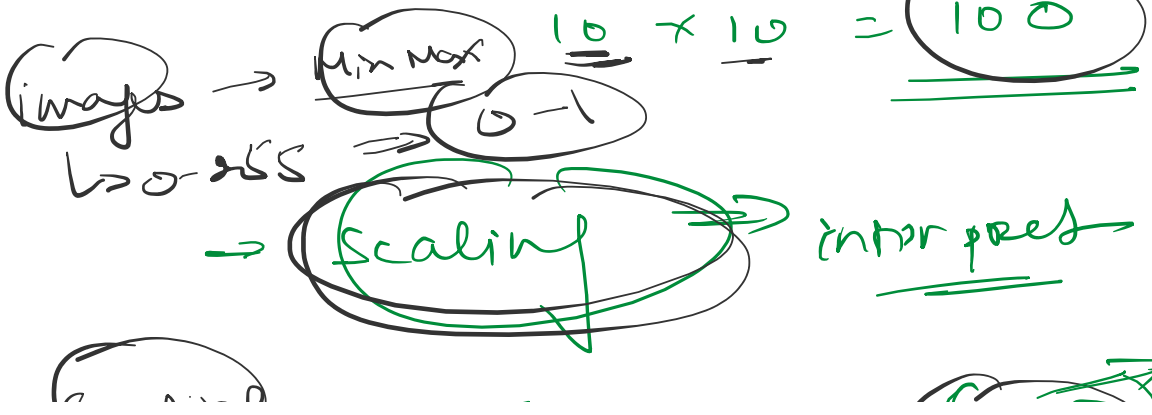
age — 0 – 100

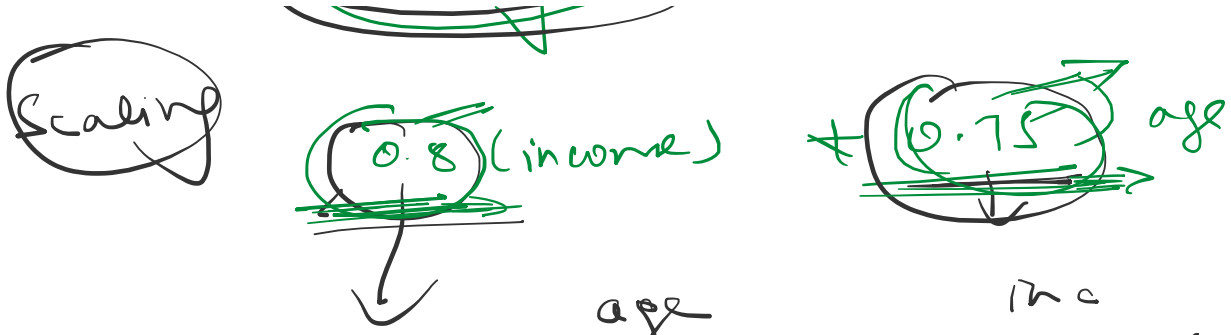
(*) $y = 0.001 (\text{income}) \times 10 (\text{Age})$

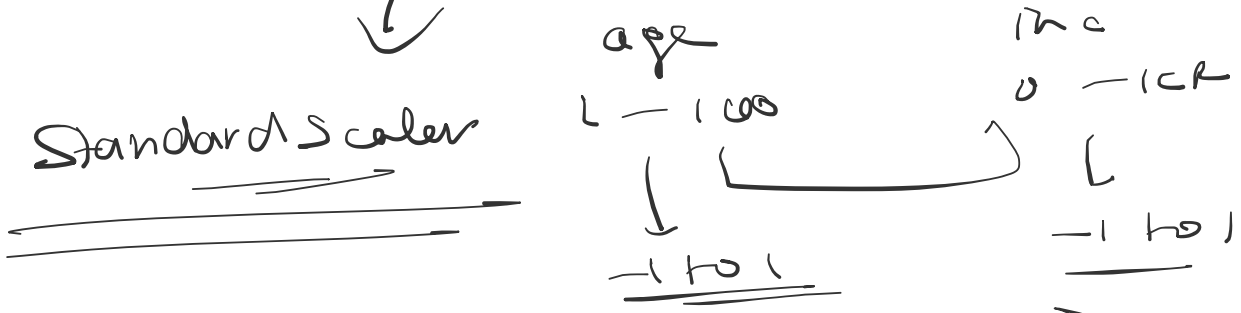
Ag ✗



Prev







(1) Standardize the Features ⇒



Std Scaler

card_type: quiz_card

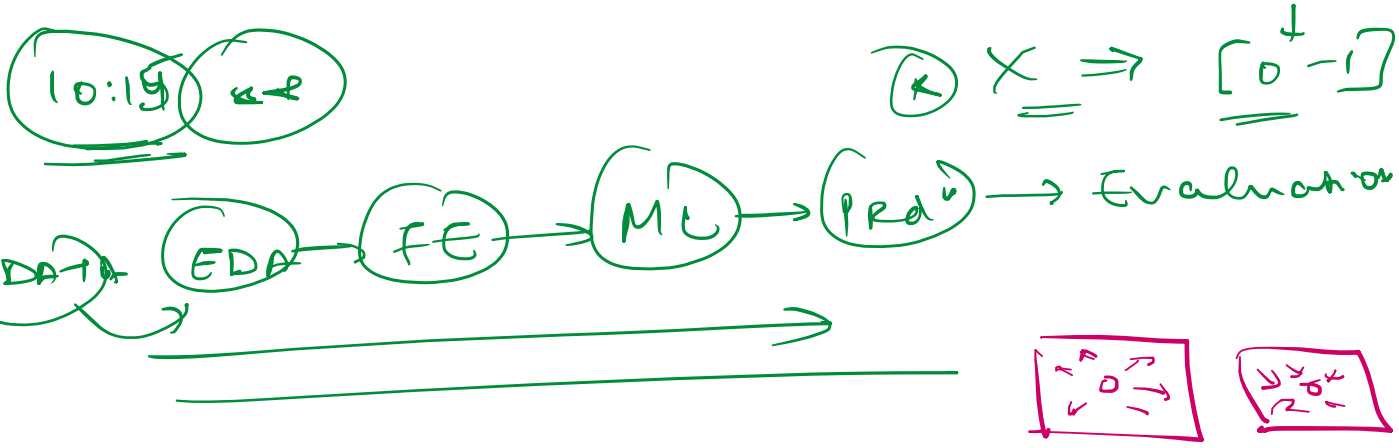
Question

When assessing model interpretability in Linear Regression, what is the impact of feature scaling?

Choices

- [ ] Feature scaling does not affect model interpretability
- [ ] Feature scaling improves model interpretability
- [x] Feature scaling can help compare the magnitudes of different coefficients

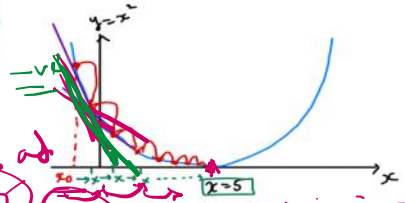
-1.41 , -0.71



Gradient Descent ⇒

→ opt. algo → minima

Gradient Descent



$L = y = f(x) = (x-5)^2 = 0$

G.D → minima ka idea

1. Pick randomly x_0
2. $\frac{\partial L}{\partial x}$ at x_0 → -ve
3. $x = x - \eta \cdot \frac{\partial L}{\partial x}$ → learning rate : 0.1

move towards neg derivatives

-ve + -ve

derivative

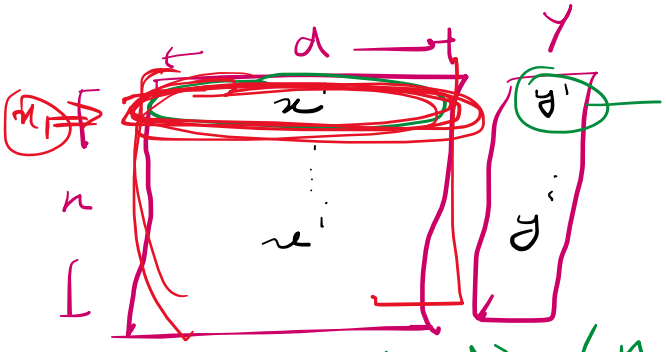
$x_0 = 2$ → -ve

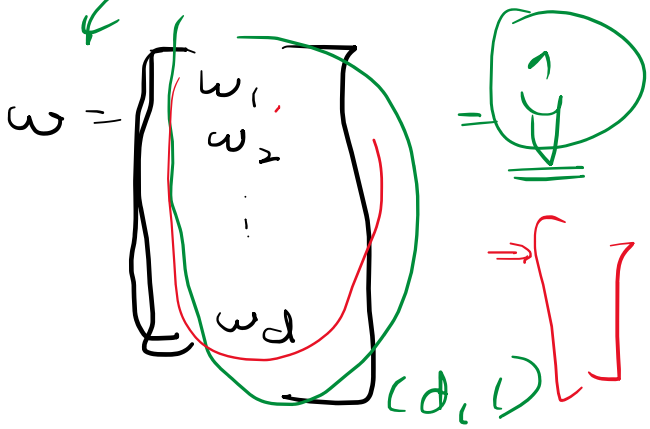
-ve × -ve → +ve

minima

LINEAR Regression ⇒

model





$x_1 \Rightarrow [$] $\hat{y}_1 = w_1^T x_1 + w_0$

$(d \times 1)$ $(n \times d)$ $(n \times d)$

pred^n for 1st DP

$\hat{y}_i = w^T x_i + w_0$

Numpy → VECTORIZATION

$\hat{y} = X \cdot w + w_0$

(n × d) (d × 1) (1)

dot

pred^n dot product

$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$ DP (n × 1)

predict

↳ dot

np.dot(X, weights)

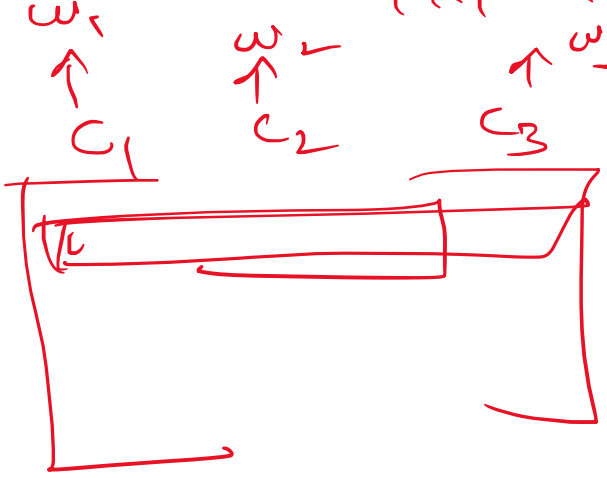


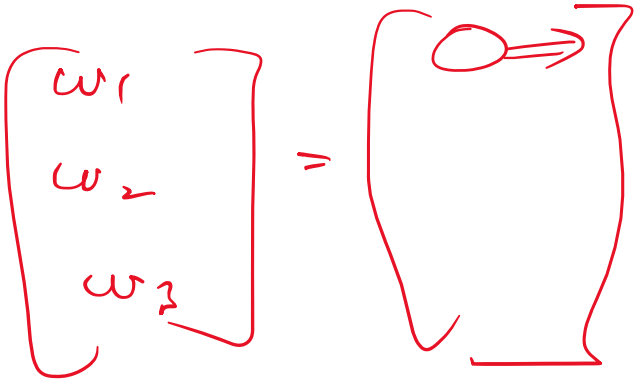




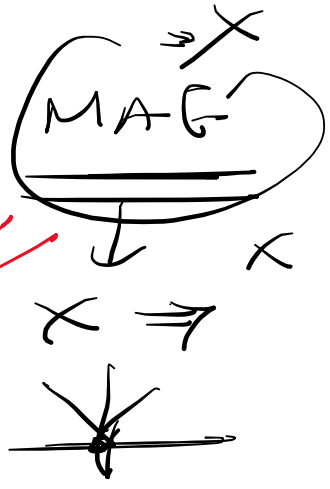


$w_1 x_1' + w_2 x_2' \quad w_3 x_2'$



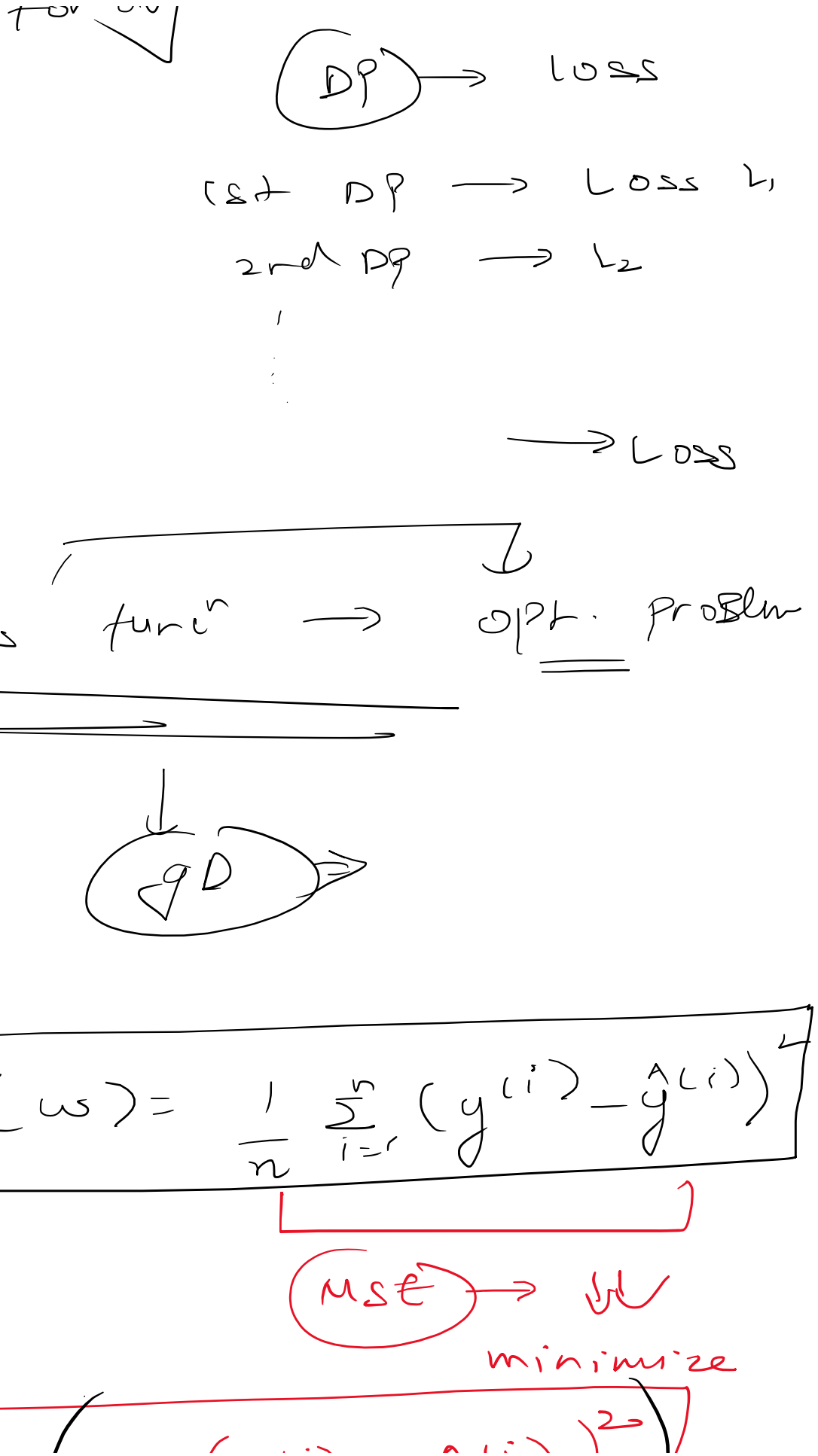


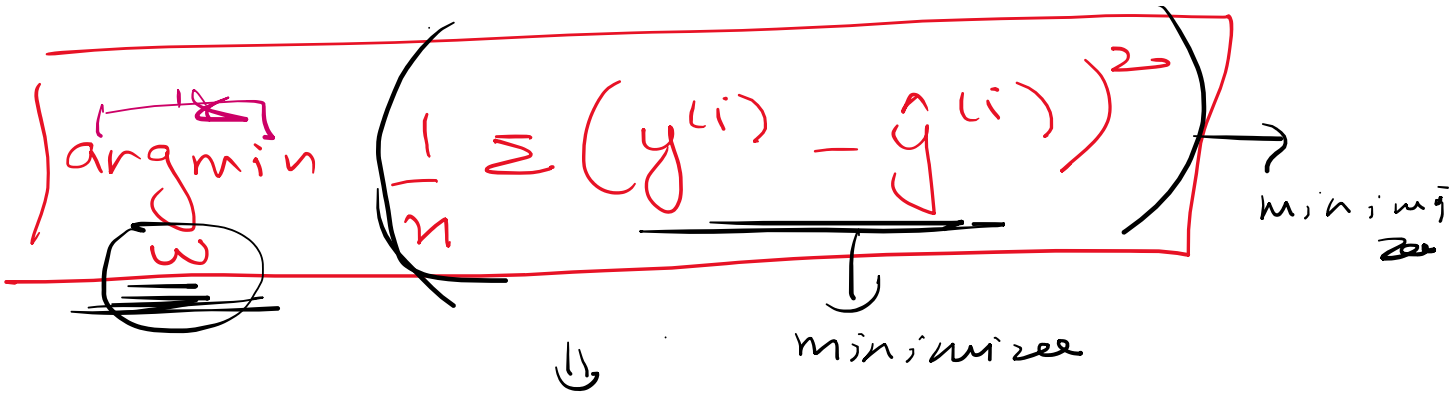
* Loss func^n ⇒ MSE & MAE



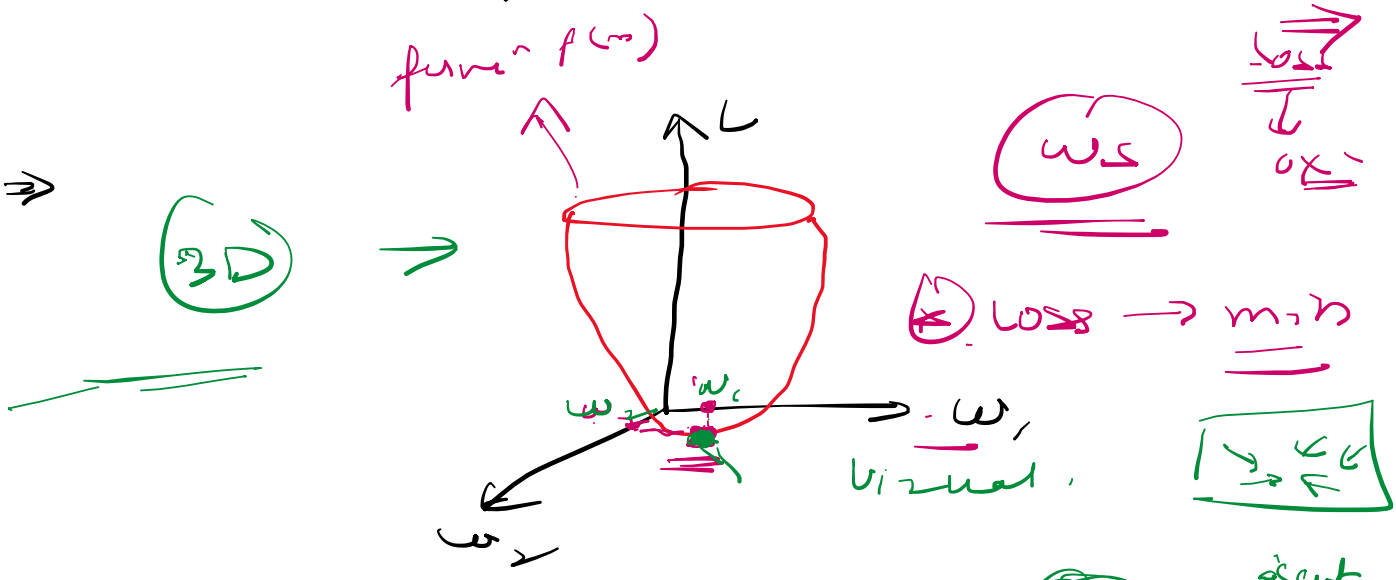
$$MSE = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)}\right)^2$$ ⇒ loss func^n

for avg


DP
loss
1st DP → Loss L1
2nd DP → L2
Loss
Loss func^n → opt. problem
GD
$Loss(w) = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})$
MSE
minimize



OPTIMIZATION PROBLEM



Question

In gradient descent, what does the gradient represent ?

Choices

- [x] The direction of steepest increase of the cost function
- [ ] The direction of steepest decrease of the cost function
- [ ] The number of training examples in the dataset
- [ ] The number of layers in the neural network



$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}$$

(*) ∴ w_d ↓

Loss is min

(4) Solving opt. problem ⇒

(1) → Randomly init w

(2) $\frac{dL}{dw}$ $\nabla_w L$

(3) Repeat { → 100 Times → 100 Times

$w_j = w_j - \eta \times \frac{dL}{dw_j}$ → 1e

}

Gradients

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \end{bmatrix}$$

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^{n} (y^i - \hat{y}^i)^2$$

go away

$$w = \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix}$$

go away

$$\frac{\partial L}{\partial w} = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_j} \\ \vdots \\ \frac{\partial L}{\partial w_d} \end{bmatrix}$$

For 1 DATA point

$$\Rightarrow \frac{\partial L}{\partial w_j} = \frac{\partial}{\partial w_j} (y - \hat{y})^2$$

chain Rule

$$= \frac{\partial (y - \hat{y})^2}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_j}$$

$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_j x_j + \cdots + w_d x_d + w_0$

w_j

$$= -2 (y - \hat{y}) \cdot x_j \Rightarrow \text{for 1 point}$$

$= -2(y - \hat{y}) \cdot x_j$ ⇒ for 1 point

$= 2(\hat{y} - y) \cdot x_j$

④ Bring summation again ⇒

derivative for all points ⇒

$$\frac{dL}{dw_j} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y}^{(i)} - y^{(i)}) \, x_j^{(i)}$$

GD → ① → Random init w

② $\frac{dL}{dw_j}$ ⇒ ✓

Skipping

③ Repeat 4

$$w_j = w_j - \eta \left[\frac{2}{n} \sum_{i=1}^{n} (\hat{y}^{(i)} - y^{(i)}) \, x_j^{(i)} \right]$$

$w_0 = w_0$

(MSE) → Opt. → Solver

